

CUSTOMER LIFETIME VALUE ANALYSIS

Industrial Training Report
Submitted in Partial fulfilment
of the Degree of
Bachelors of Computer Applications

Submitted By:-Anurag Kumar

Enrollment No.- 120924143

Sem- V



Jagan Nath University

Bahadurgarh (NCR)

(2024-27)

CUSTOMER LIFETIME VALUE ANALYSIS

Industrial Training Report
Submitted in Partial fulfilment
of the Degree of
Bachelors of Computer Applications

Supervisor's Name

Guide 1: Ms Anima Sinha

Guide 2: Dr Madhu Chauhan

Supervisor's Signature :

Submitted By:-Anurag Kumar

Enrollment No.- 120924143

Sem- V

Student's Signature :



Jagan Nath University
Bahadurgarh (NCR)
(2024-27)

Project Certification

This is to certify that the Industrial Training Report entitled “**Customer Lifetime Value Analysis**” has been prepared and submitted by **Anurag Kumar**, a student of **Bachelor of Computer Applications (BCA), Jagan Nath University, Bahadurgarh (NCR)**, as a part of the academic requirements for the session **2024–27**. The report presents the knowledge, skills, and practical experience gained by the student during the industrial training period.

The work presented in this report describes the data science training activities and project work carried out using **Python, data analysis, machine learning, data visualization, and Streamlit**.

Name of the Student:

Anurag Kumar

Name of the Guide:

Guide 1: Ms Anima Sinha

Guide 2: Dr Madhu Chauhan

Enrollment NO.: 120924143

Date: 10 September 2026

Student’s Signature:

Guide Signature:

CERTIFICATE OF COMPLETION



ACKNOWLEDGEMENT

I would like to sincerely express my gratitude to my guides, **Ms. Anima Sinha and Dr. Madhu Chauhan**, for their valuable guidance, continuous support, and constructive suggestions throughout my training and the completion of this project. Their encouragement and guidance helped me gain a better understanding of the practical aspects of Data Science and improve my technical knowledge and skills.

I am also thankful to **Jagan Nath University, Bahadurgarh (NCR)** for providing me with a supportive academic environment, resources, and opportunities that helped me successfully complete my training.

I am grateful for the opportunity to work on the project “**Customer Lifetime Value Analysis.**” Working on this project provided me with practical experience in **Python, data analysis, data cleaning, exploratory data analysis, data visualization, RFM analysis, customer segmentation, machine learning, and Streamlit.** This experience helped me connect theoretical concepts with their practical applications.

Finally, I would like to thank all my teachers, classmates, and everyone who supported and encouraged me throughout the training. Their cooperation and motivation played an important role in helping me complete this project successfully.

INDEX

S. No.	Contents	Page No.
1	Hardware / Software Requirements	6
2	Introduction to Data Science	7
3	List of Practicals – with Screenshots 3.1 Data Loading and Initial Understanding 3.2 Data Cleaning and Preprocessing 3.3 Exploratory Data Analysis 3.4 Feature Engineering 3.5 RFM Analysis 3.6 Customer Segmentation 3.7 Machine Learning Approach 3.8 Train-Test Split and Model Training 3.9 Model Evaluation 3.10 Feature Importance 3.11 Customer Segment Analysis 3.12 Streamlit Prediction Interface	8
4	Project Code & Screenshots	20
5	References	41

1. Hardware / Software Requirements

- Hardware: Computer/Laptop with an internet connection.
- Operating System: Windows. Programming
- Language: Python.
- Development Environment: Jupyter Notebook.
- Libraries/Tools:
 - i. NumPy
 - ii. Pandas
 - iii. Matplotlib
 - iv. Seaborn
 - v. Scikit-learn
 - vi. Joblib
 - vii. Streamlit
- Web Browser: Used to run and access the Streamlit application.

2. INTRODUCTION TO DATA SCIENCE

Data Science is an interdisciplinary field that combines programming, statistics, data analysis, visualization, and machine learning to extract useful information from data. It helps organizations understand patterns, identify trends, and make better decisions. In this project, Data Science techniques are applied to retail transaction data to study customer purchasing behaviour and understand the value of different customers. The project mainly focuses on RFM (Recency, Frequency, and Monetary) analysis, which helps measure how recently customers purchased, how frequently they purchased, and how much they spent.

The project follows a practical Data Science workflow that includes data loading, cleaning, preprocessing, exploratory data analysis, visualization, feature engineering, customer segmentation, and machine learning. Based on RFM values, customers are divided into segments such as Champions, Loyal Customers, Potential Loyalists, At Risk, and Lost Customers. A Random Forest Classifier is then trained using Recency, Frequency, and Monetary features to predict customer segments. The project also includes a simple Streamlit application where users can enter customer values and obtain a predicted segment. Overall, the project provides practical experience in data analysis, customer segmentation, machine learning, model evaluation, and deployment.

3. LIST OF PRACTICALS – WITH SCREENSHOTS

Data Loading and Initial Understanding

The Online Retail dataset is loaded using Pandas. The first and last records, samples, dimensions, data types, columns, descriptive statistics, missing values, and duplicate records are checked.

[1]:

```
#libraries
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:

```
#Load the dataset
df = pd.read_excel("Online_Retail.xlsx")
```

[3]:

```
df
```

[3]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	12680.0	France

541909 rows x 8 columns

[10]:

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description     540455 non-null object
3   Quantity        541909 non-null int64
4   InvoiceDate     541909 non-null datetime64[ns]
5   UnitPrice       541909 non-null float64
6   CustomerID     406829 non-null float64
7   Country         541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
```

[11]:

```
df['CustomerID'] = df['CustomerID'].astype('Int64')
df['CustomerID'].dtypes
```

[11]:

```
Int64Dtype()
```

[12]:

```
df.describe()
```

[12]:

	Quantity	InvoiceDate	UnitPrice	CustomerID
count	541909.000000	541909	541909.000000	406829.0
mean	9.552250	2011-07-04 13:34:57.156386048	4.611114	15287.69057
min	-80995.000000	2010-12-01 08:26:00	-11062.060000	12346.0
25%	1.000000	2011-03-28 11:34:00	1.250000	13953.0
50%	3.000000	2011-07-19 17:17:00	2.080000	15152.0
75%	10.000000	2011-10-19 11:27:00	4.130000	16791.0
max	80995.000000	2011-12-09 12:50:00	38970.000000	18287.0
std	218.081158	NaN	96.759853	1713.600303

Data Cleaning and Preprocessing

Missing CustomerID records and duplicate transactions are removed. Invalid Quantity and UnitPrice values are filtered, InvoiceDate is converted to datetime, and TotalAmount is created.

```
[13]: # Checking for missing values
      df.isnull().sum()

[13]: InvoiceNo      0
      StockCode    0
      Description  1454
      Quantity     0
      InvoiceDate   0
      UnitPrice    0
      CustomerID  135080
      Country      0
      dtype: int64

[14]: df.duplicated().sum()

[14]: np.int64(5268)

[15]: #Dropping rows with missing CustomerID
      df=df.dropna(subset=['CustomerID'])

[16]: df.shape

[16]: (406829, 8)

[17]: df=df.drop_duplicates() # duplicate value removed

[18]: df.shape

[18]: (401604, 8)

[19]: df=df[df['Quantity']>0] #-ve qty removed
```

```
[20]: df['Quantity'].describe()

[20]: count    392732.000000
      mean      13.153718
      std      181.588420
      min       1.000000
      25%       2.000000
      50%       6.000000
      75%      12.000000
      max      80995.000000
      Name: Quantity, dtype: float64

[21]: df=df[df['UnitPrice']>0] #-ve price removed

[22]: # Generate summary statistics of the UnitPrice column.
      df['UnitPrice'].describe()

[22]: count    392692.000000
      mean     3.125914
      std     22.241836
      min     0.001000
      25%     1.250000
      50%     1.950000
      75%     3.750000
      max    8142.750000
      Name: UnitPrice, dtype: float64

[23]: # Creating a new column for total price
      df['TotalAmount'] = df['Quantity'] * df['UnitPrice']

[24]: df[['Quantity', 'UnitPrice', 'TotalAmount']].head()

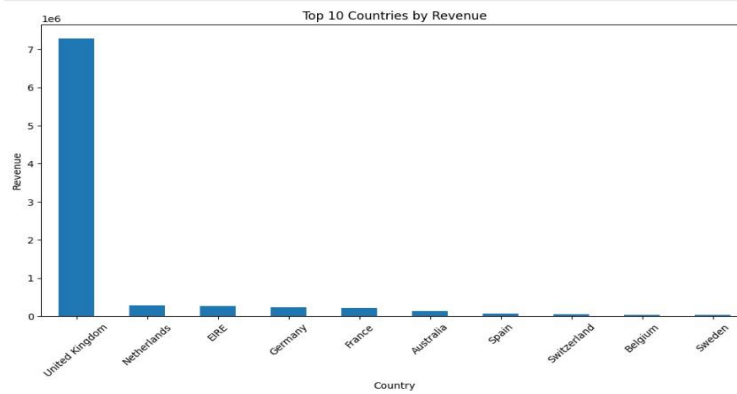
[24]:
```

	Quantity	UnitPrice	TotalAmount
0	6	2.55	15.30
1	6	3.39	20.34
2	8	2.75	22.00
3	6	3.39	20.34
4	6	3.39	20.34

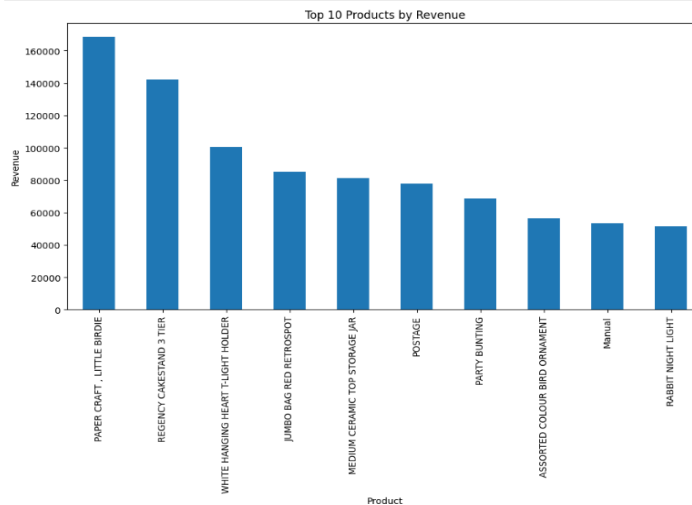
Exploratory Data Analysis

EDA is performed to understand transaction distributions and business patterns, including country revenue, product revenue, transaction amounts, and customer purchasing behaviour.

```
[35]: plt.figure(figsize=(12,6))
country_revenue.head(10).plot(kind='bar')
plt.title("Top 10 Countries by Revenue")
plt.xlabel("Country")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.show()
```



```
[37]: plt.figure(figsize=(12,6))
product_revenue.head(10).plot(kind='bar')
plt.title("Top 10 Products by Revenue")
plt.xlabel("Product")
plt.ylabel("Revenue")
plt.xticks(rotation=90)
plt.show()
```



Feature Engineering

Feature engineering converts raw transaction records into useful customer-level variables. TotalAmount is calculated first. Transactions are then aggregated by CustomerID to calculate customer revenue, purchase frequency, recency, and average order value.

```
[41]: # top customers
customer_revenue = (
    df.groupby('CustomerID')['TotalAmount']
      .sum()
      .sort_values(ascending=False)
)
```

```
[42]: customer_revenue.head(10)
```

```
[42]: CustomerID
14646    280206.02
18102    259657.30
17450    194390.79
16446    168472.50
14911    143711.17
12415    124914.53
14156    117210.08
17511     91062.38
16029     80850.84
12346     77183.60
Name: TotalAmount, dtype: float64
```

```
[43]: plt.figure(figsize=(12,6))

customer_revenue.head(10).plot(kind='bar')

plt.title("Top 10 Customers by Revenue")

plt.xlabel("Customer ID")

plt.ylabel("Revenue")

plt.xticks(rotation=45)

plt.show()
```

RFM Analysis

RFM stands for Recency, Frequency, and Monetary. Recency measures how recently a customer purchased; Frequency measures how often the customer purchased; and Monetary measures how much the customer spent.

The implementation uses quantile-based scoring with pandas `qcut`. Recency is assigned scores so that more recent customers receive higher scores. Frequency and Monetary are also converted into five score levels.

```
[48]: recency = (  
    df.groupby('CustomerID')['InvoiceDate']  
    .max()  
)
```

```
[49]: recency = (snapshot_date - recency).dt.days
```

```
[50]: recency.head()
```

```
[50]: CustomerID  
12346    326  
12347     2  
12348    75  
12349    19  
12350   310  
Name: InvoiceDate, dtype: int64
```

```
[51]: rfm = pd.DataFrame({  
    'Recency': recency,  
    'Frequency': purchase_frequency,  
    'Monetary': customer_revenue  
})
```

```
[52]: rfm.head()
```

```
[52]:
```

	Recency	Frequency	Monetary
CustomerID			
12346	326	1	77183.60
12347	2	7	4310.00
12348	75	4	1797.24
12349	19	1	1757.55
12350	310	1	334.40

```
[53]: rfm['R_Score'] = pd.qcut(  
    rfm['Recency'],  
    q=5,  
    labels=[5,4,3,2,1]  
)
```

```
[54]: rfm['F_Score'] = pd.qcut(  
    rfm['Frequency'],  
    q=5,  
    labels=False,  
    duplicates='drop'  
    ) + 1
```

Customer Segmentation

The `customer_segment` function assigns business segments based on the RFM scores. The implemented rules are:

RFM Rule	Segment
$R_Score \geq 4$ and $F_Score \geq 4$	Champions
$R_Score \geq 3$ and $F_Score \geq 3$	Loyal Customers
$R_Score \geq 4$ and $F_Score \leq 2$	Potential Loyalists
$R_Score \leq 2$ and $F_Score \geq 3$	At Risk
All remaining cases	Lost Customers

```
[61]: rfm.head()
```

```
[61]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CustomerID							
12346	326	1	77183.60	1	1	5	115
12347	2	7	4310.00	5	4	5	545
12348	75	4	1797.24	2	3	4	234
12349	19	1	1757.55	4	1	4	414
12350	310	1	334.40	1	1	2	112

```
[62]: def customer_segment(row):  
  
    if row['R_Score'] >= 4 and row['F_Score'] >= 4:  
        return 'Champions'  
  
    elif row['R_Score'] >= 3 and row['F_Score'] >= 3:  
        return 'Loyal Customers'  
  
    elif row['R_Score'] >= 4 and row['F_Score'] <= 2:  
        return 'Potential Loyalists'  
  
    elif row['R_Score'] <= 2 and row['F_Score'] >= 3:  
        return 'At Risk'  
  
    else:  
        return 'Lost Customers'
```

```
[63]: rfm['Segment'] = rfm.apply(customer_segment, axis=1)
```

```
[64]: rfm.head()
```

```
[64]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score	Segment
CustomerID								
12346	326	1	77183.60	1	1	5	115	Lost Customers
12347	2	7	4310.00	5	4	5	545	Champions
12348	75	4	1797.24	2	3	4	234	At Risk
12349	19	1	1757.55	4	1	4	414	Potential Loyalists
12350	310	1	334.40	1	1	2	112	Lost Customers

Machine Learning Approach

Supervised learning is used in this project because each customer is assigned a **Segment** based on the RFM analysis and business rules. The machine learning model learns from these customer details and predicts the appropriate customer segment.

A **Random Forest Classifier** is used for prediction. It combines multiple decision trees and can handle different patterns in customer data. It also provides feature importance, which helps in understanding which features have the most effect on the prediction.

```
[76]: rfm['Average_Order_Value']
```

```
[76]: CustomerID
12346    77183.600000
12347     615.714286
12348     449.310000
12349    1757.550000
12350     334.400000
...
18280    180.600000
18281      80.820000
18282      89.025000
18283    127.845625
18287     612.426667
Name: Average_Order_Value, Length: 4338, dtype: float64
```

```
[77]: #MACHINE LEARNING
# Features (Input)
X = rfm[['Recency', 'Frequency', 'Monetary']]

# Target (Output)
y = rfm['Segment']
```

```
[78]: X.head()
```

```
[78]:
```

	Recency	Frequency	Monetary
CustomerID			
12346	326	1	77183.60
12347	2	7	4310.00
12348	75	4	1797.24
12349	19	1	1757.55
12350	310	1	334.40

Train-Test Split and Model Training

The dataset is split into training and testing sets using `train_test_split` with `test_size=0.2`, `random_state=42`, and `stratify=y`. The Random Forest Classifier is trained on the training data and evaluated on the unseen test data.

```
[84]: from sklearn.model_selection import train_test_split

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

```
[85]: from sklearn.ensemble import RandomForestClassifier

# Create the model
rf_model = RandomForestClassifier(
    n_estimators=100, # it include 100 decision tree
    random_state=42
)

# Train the model
rf_model.fit(X_train, y_train)
```

```
[85]: ▼ RandomForestClassifier ⓘ ?
      ► Parameters
      ► Fitted attributes
```

```
[86]: print(rf_model)

RandomForestClassifier(random_state=42)
```

```
[87]: # Predict customer segments
y_pred = rf_model.predict(X_test)
```

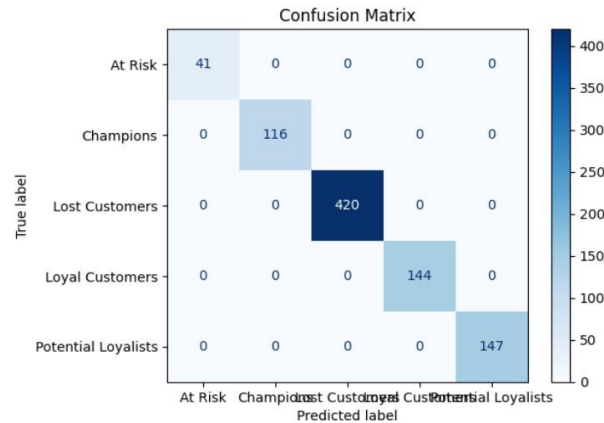
Model Evaluation

The trained model is evaluated using Accuracy, Precision, Recall, F1-Score, and a confusion matrix. The submitted execution reports 100% for the displayed classification metrics.

```
[95]: from sklearn.metrics import ConfusionMatrixDisplay
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=label_encoder.classes_
)

disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```



```
[96]: from sklearn.metrics import classification_report
```

```
print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_
))
```

	precision	recall	f1-score	support
At Risk	1.00	1.00	1.00	41
Champions	1.00	1.00	1.00	116
Lost Customers	1.00	1.00	1.00	420
Loyal Customers	1.00	1.00	1.00	144
Potential Loyalists	1.00	1.00	1.00	147
accuracy			1.00	868
macro avg	1.00	1.00	1.00	868
weighted avg	1.00	1.00	1.00	868

```
[97]: # Get feature importance
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})
```

```
# Sort from highest to lowest
importance = importance.sort_values(
    by='Importance',
    ascending=False
)

print(importance)
```

	Feature	Importance
0	Recency	0.452045
1	Frequency	0.415430
2	Monetary	0.132524

Feature Importance

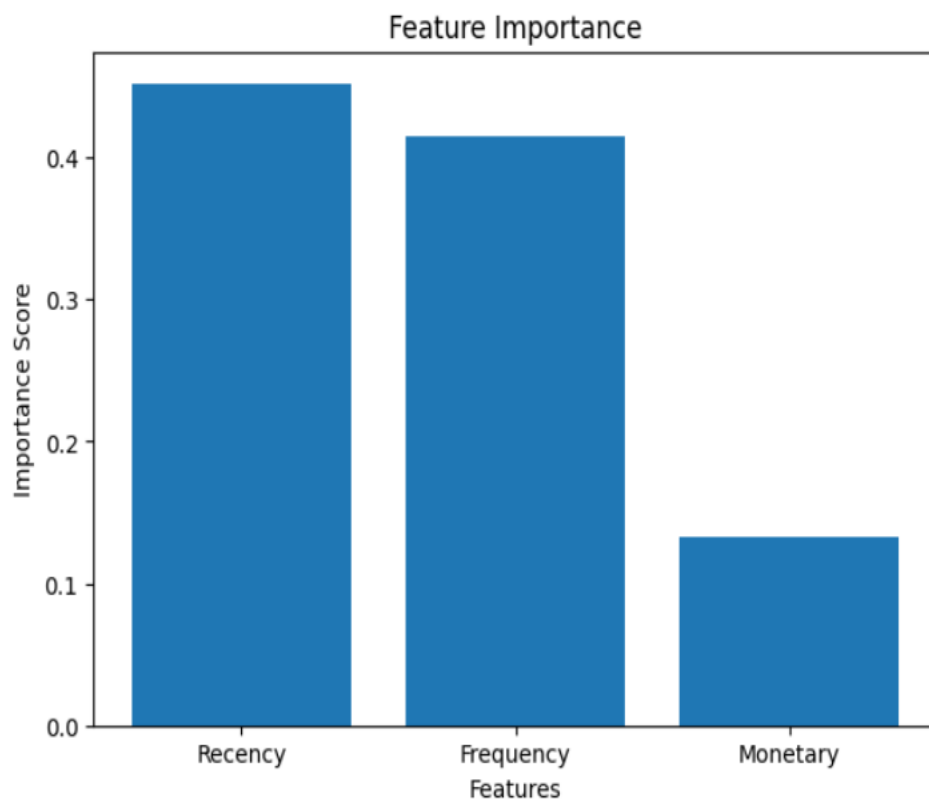
The Random Forest model provides `feature_importances_` for Recency, Frequency, and Monetary. The submitted feature-importance visualization shows Recency as the strongest of the three displayed model features, followed by Frequency and Monetary.

```
[98]: plt.figure(figsize=(7,5))

plt.bar(
    importance['Feature'],
    importance['Importance']
)

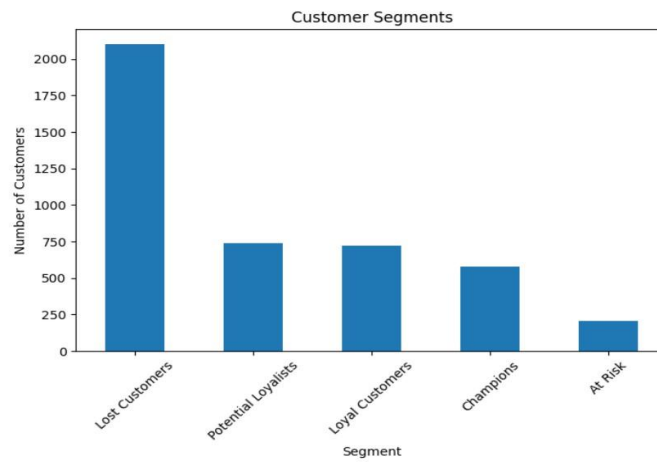
plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance Score")

plt.show()
```

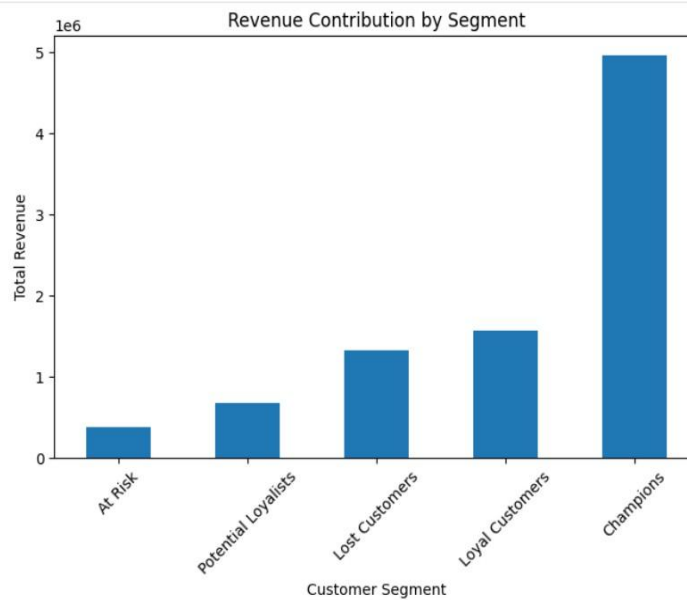


Customer Segment Analysis

The project output reports customer counts for Lost Customers, Potential Loyalists, Loyal Customers, Champions, and At Risk segments.



```
[67]: segment_summary = rfm.groupby('Segment').agg(  
    Avg_Recency=('Recency', 'mean'),  
    Avg_Frequency=('Frequency', 'mean'),  
    Avg_Monetary=('Monetary', 'mean'),  
    Customer_Count=('Recency', 'count')  
)  
segment_summary
```



```
[75]: rfm['Average_Order_Value'] = rfm['Monetary'] / rfm['Frequency']
```

Streamlit Deployment

The project includes a Streamlit interface for interactive customer-segment prediction. The application loads the saved model and label encoder using Joblib. The user provides Recency, Frequency, and Monetary values through numeric inputs, and the application displays the predicted customer segment.

```
app.py > ...
3  import streamlit as st
4
5  # Load the model and preprocessors using joblib
6  model = joblib.load("rf_model.pkl")
7  label_encoder = joblib.load("label_encoder.pkl")
8  features = joblib.load("feature_columns.pkl")
9
10 st.title("Customer Lifetime Value Analysis")
11
12 recency = st.number_input("Recency", 0, 1000, 8)
13 frequency = st.number_input("Frequency", 0, 1000, 12)
14 monetary = st.number_input("Monetary", 0.0, 1000000.0, 4200.0)
15
16 if st.button("Predict Segment"):
17     df = pd.DataFrame([[recency, frequency, monetary]], columns=features)
18     pred = model.predict(df)
19     seg = label_encoder.inverse_transform(pred)[0]
20     st.success(f"Predicted Segment: {seg}")
```

Customer Lifetime Value Analysis

Recency

4 - +

Frequency

6 - +

Monetary

1200.00 - +

Predict Segment

Predicted Segment: Loyal Customers

4. Project Code and Screenshots

The following section preserves the project/code screenshots supplied with the original project documentation. They are included as the implementation evidence and are grouped in the same order as the submitted code document. Each figure represents a code, analysis, model, or output stage from the project.

```
[1]: #libraries
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

[2]: #load the dataset
df = pd.read_excel("Online_Retail.xlsx")

[3]: df
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	844068	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	12680.0	France

541909 rows x 8 columns

```
[4]: df.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	844068	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

```
[5]: df.tail()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	12680.0	France

```
[6]: df.sample(7)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
	386640	570241	90176E	DIAMANTE NECKLACE GREEN	1	2011-10-09 15:24:00	7.50	14801.0 United Kingdom
	274154	560885	22553	PLASTERS IN TIN SKULLS	24	2011-07-21 16:20:00	1.65	16188.0 United Kingdom
	319427	564855	23108	SET OF 10 LED DOLLY LIGHTS	4	2011-08-31 09:10:00	6.25	16746.0 United Kingdom
	174245	551776	82600	NO SINGING METAL SIGN	5	2011-05-04 11:23:00	2.10	13268.0 United Kingdom
	205211	554756	84821	DANISH ROSE DELUXE COASTER	12	2011-05-26 11:36:00	0.85	16837.0 United Kingdom
	386274	570230	21684	SMALL MEDINA STAMPED METAL BOWL	12	2011-10-09 13:41:00	0.85	18088.0 United Kingdom
	295114	562772	22423	REGENCY CAKESTAND 3 TIER	2	2011-08-09 12:07:00	12.75	13097.0 United Kingdom

```
[7]: df.shape
[7]: (541909, 8)
[8]: df.dtypes
[8]: InvoiceNo      object
StockCode      object
Description     object
Quantity       int64
InvoiceDate    datetime64[ns]
UnitPrice      float64
CustomerID     float64
Country        object
dtype: object
[9]: df.columns
[9]: Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
        'UnitPrice', 'CustomerID', 'Country'],
        dtype='object')

[10]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  --
0   InvoiceNo    541909 non-null  object
1   StockCode   541909 non-null  object
2   Description  540455 non-null  object
3   Quantity    541909 non-null  int64
4   InvoiceDate  541909 non-null  datetime64[ns]
5   UnitPrice   541909 non-null  float64
6   CustomerID  406829 non-null  float64
7   Country     541909 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB

[11]: df['CustomerID'] = df['CustomerID'].astype('Int64')
df['CustomerID'].dtypes
[11]: Int64Dtype()

[12]: df.describe()
[12]:
```

	Quantity	InvoiceDate	UnitPrice	CustomerID
count	541909.000000	541909	541909.000000	406829.0
mean	9.552250	2011-07-04 13:34:57.156386048	4.611114	15287.69057
min	-80995.000000	2010-12-01 08:26:00	-11062.060000	12346.0
25%	1.000000	2011-03-28 11:34:00	1.250000	13953.0
50%	3.000000	2011-07-19 17:17:00	2.080000	15152.0
75%	10.000000	2011-10-19 11:27:00	4.130000	16791.0
max	80995.000000	2011-12-09 12:50:00	38970.000000	18287.0
std	218.081158	NaN	96.759853	1713.600303

```
[13]: # Checking for missing values
df.isnull().sum()

[13]: InvoiceNo      0
      StockCode    0
      Description  1454
      Quantity     0
      InvoiceDate   0
      UnitPrice    0
      CustomerID  135080
      Country      0
      dtype: int64

[14]: df.duplicated().sum()

[14]: np.int64(5268)

[15]: #Dropping rows with missing CustomerID
df=df.dropna(subset=['CustomerID'])

[16]: df.shape

[16]: (406829, 8)

[17]: df=df.drop_duplicates() # duplicate value removed

[18]: df.shape

[18]: (401604, 8)

[19]: df=df[df['Quantity']>0] #-ve qty removed

[20]: df['Quantity'].describe()

[20]: count    392732.000000
      mean      13.153718
      std      181.588420
      min        1.000000
      25%        2.000000
      50%        6.000000
      75%       12.000000
      max      80995.000000
      Name: Quantity, dtype: float64

[21]: df=df[df['UnitPrice']>0] #-ve price removed

[22]: # Generate summary statistics of the UnitPrice column.
df['UnitPrice'].describe()

[22]: count    392692.000000
      mean      3.125914
      std      22.241836
      min       0.001000
      25%       1.250000
      50%       1.950000
      75%       3.750000
      max      8142.750000
      Name: UnitPrice, dtype: float64

[23]: # Creating a new column for total price
df['TotalAmount'] = df['Quantity'] * df['UnitPrice']

[24]: df[['Quantity', 'UnitPrice', 'TotalAmount']].head()

[24]:
```

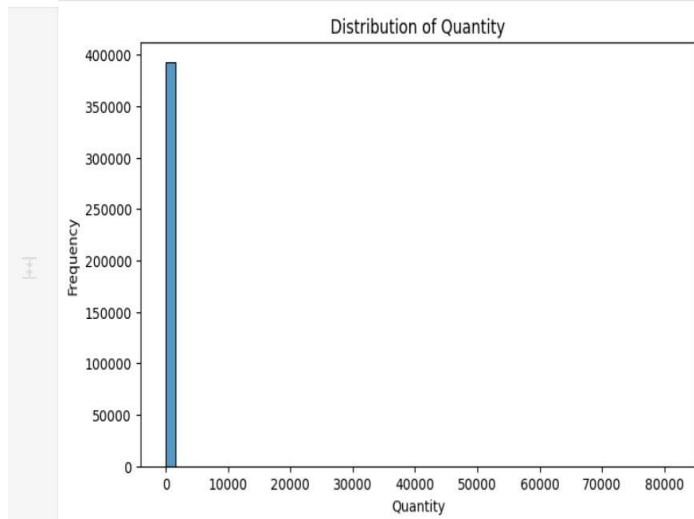
	Quantity	UnitPrice	TotalAmount
0	6	2.55	15.30
1	6	3.39	20.34
2	8	2.75	22.00
3	6	3.39	20.34
4	6	3.39	20.34

```
[25]: plt.figure(figsize=(8,5))

sns.histplot(df['Quantity'], bins=50)

plt.title("Distribution of Quantity")
plt.xlabel("Quantity")
plt.ylabel("Frequency")

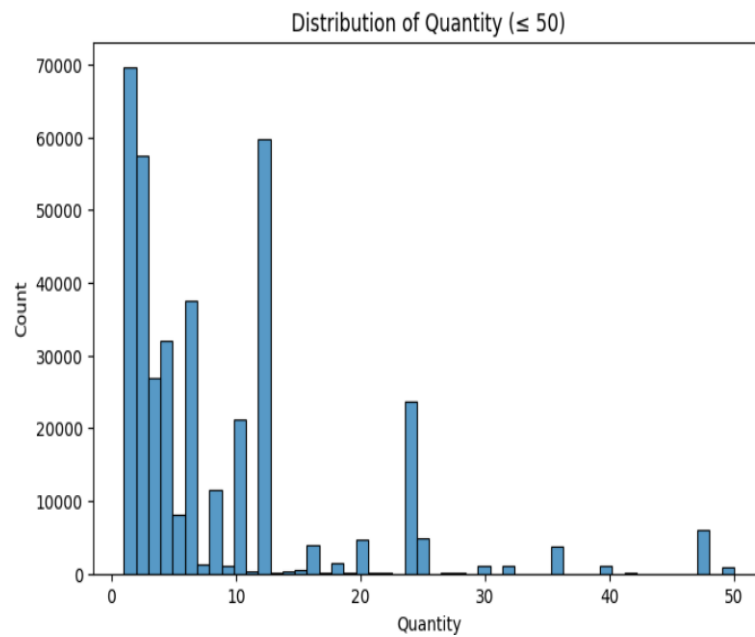
plt.show()
```



```
[26]: plt.figure(figsize=(8,5))

sns.histplot(df[df['Quantity'] <= 50]['Quantity'], bins=50)

plt.title("Distribution of Quantity (≤ 50)")
plt.show()
```

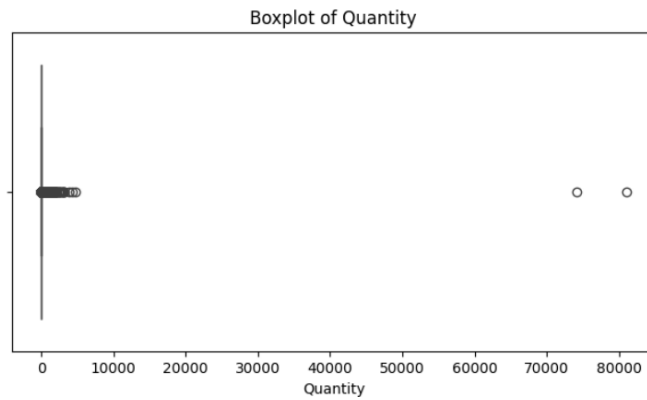


```
[27]: plt.figure(figsize=(8,4))

sns.boxplot(x=df['Quantity'])

plt.title("Boxplot of Quantity")

plt.show()
```



```
[28]: plt.figure(figsize=(8,5))

sns.histplot(df['UnitPrice'], bins=50)

plt.title("Distribution of Unit Price")
plt.xlabel("Unit Price")
plt.ylabel("Frequency")

plt.show()
```

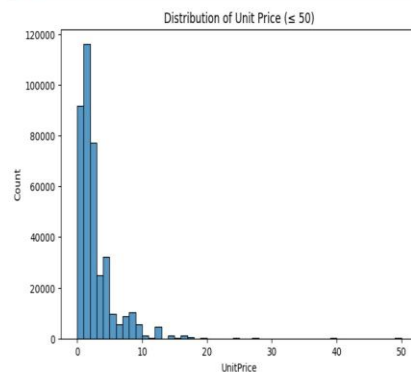


```
[29]: plt.figure(figsize=(8,5))

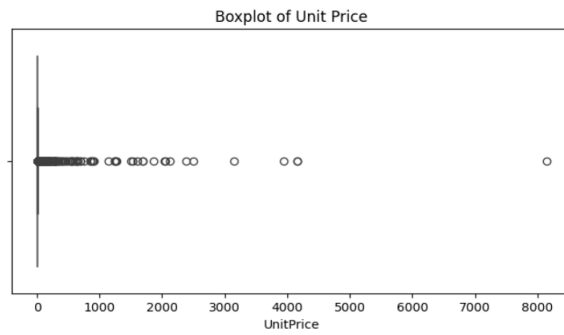
sns.histplot(df[df['UnitPrice'] <= 50]['UnitPrice'], bins=50)

plt.title("Distribution of Unit Price (<= 50)")

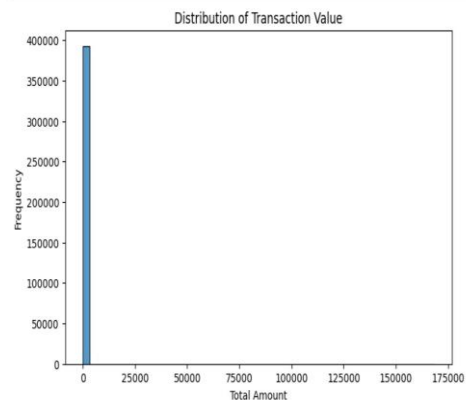
plt.show()
```



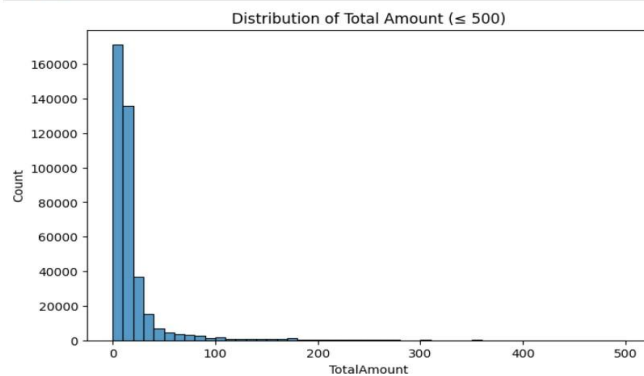
```
[30]: plt.figure(figsize=(8,4))
sns.boxplot(x=df['UnitPrice'])
plt.title("Boxplot of Unit Price")
plt.show()
```



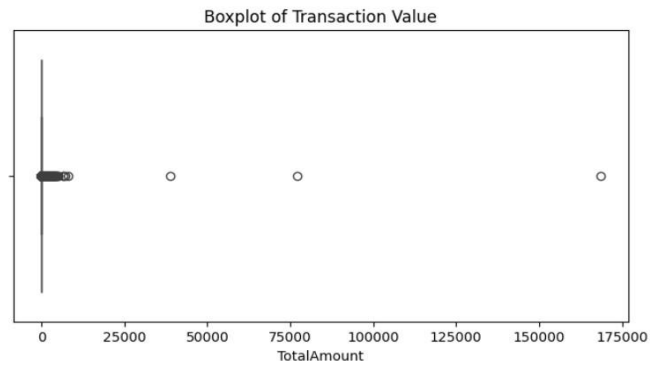
```
[31]: plt.figure(figsize=(8,5))
sns.histplot(df['TotalAmount'], bins=50)
plt.title("Distribution of Transaction Value")
plt.xlabel("Total Amount")
plt.ylabel("Frequency")
plt.show()
```



```
[32]: plt.figure(figsize=(8,5))
sns.histplot(df[df['TotalAmount'] <= 500]['TotalAmount'], bins=50)
plt.title("Distribution of Total Amount (≤ 500)")
plt.show()
```



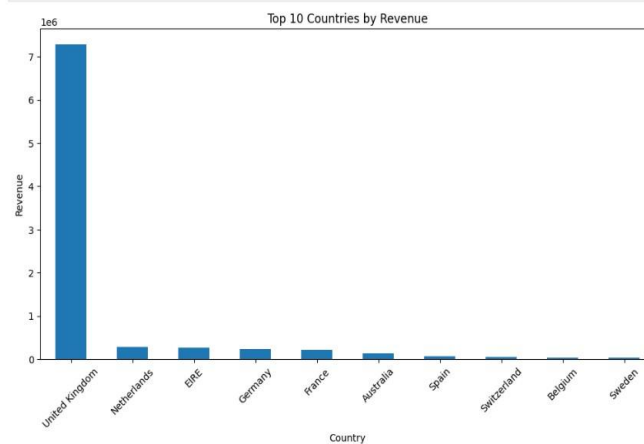
```
[33]: plt.figure(figsize=(8,4))
      sns.boxplot(x=df['TotalAmount'])
      plt.title("Boxplot of Transaction Value")
      plt.show()
```



```
[34]: country_revenue = (
      df.groupby('Country')['TotalAmount']
        .sum()
        .sort_values(ascending=False)
      )
      country_revenue.head(10)
```

```
[34]: Country
United Kingdom    7285024.644
Netherlands       285446.340
EIRE              265262.460
Germany           228678.400
France            208934.310
Australia         138453.810
Spain             61558.560
Switzerland       56443.950
Belgium           41196.340
Sweden            38367.830
Name: TotalAmount, dtype: float64
```

```
[35]: plt.figure(figsize=(12,6))
      country_revenue.head(10).plot(kind='bar')
      plt.title("Top 10 Countries by Revenue")
      plt.xlabel("Country")
      plt.ylabel("Revenue")
      plt.xticks(rotation=45)
      plt.show()
```



```
[36]: product_revenue = (
        df.groupby('Description')['TotalAmount']
          .sum()
          .sort_values(ascending=False)
      )
product_revenue.head(10)
```

```
[36]: Description
PAPER CRAFT , LITTLE BIRDIE      168469.60
REGENCY CAKESTAND 3 TIER        142264.75
WHITE HANGING HEART T-LIGHT HOLDER 100392.10
JUMBO BAG RED RETROSPOT         85040.54
MEDIUM CERAMIC TOP STORAGE JAR   81416.73
POSTAGE                          77803.96
PARTY BUNTING                   68785.23
ASSORTED COLOUR BIRD ORNAMENT    56413.03
Manual                           53419.93
RABBIT NIGHT LIGHT              51251.24
Name: TotalAmount, dtype: float64
```

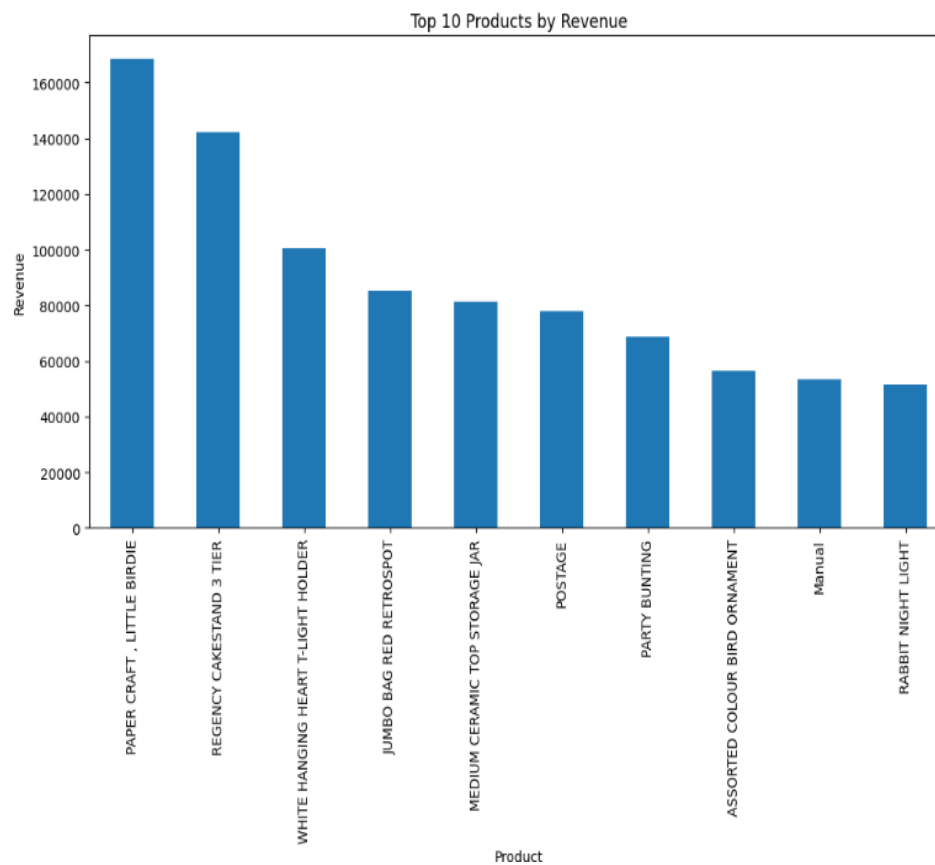
```
[37]: plt.figure(figsize=(12,6))

product_revenue.head(10).plot(kind='bar')

plt.title("Top 10 Products by Revenue")
plt.xlabel("Product")
plt.ylabel("Revenue")

plt.xticks(rotation=90)

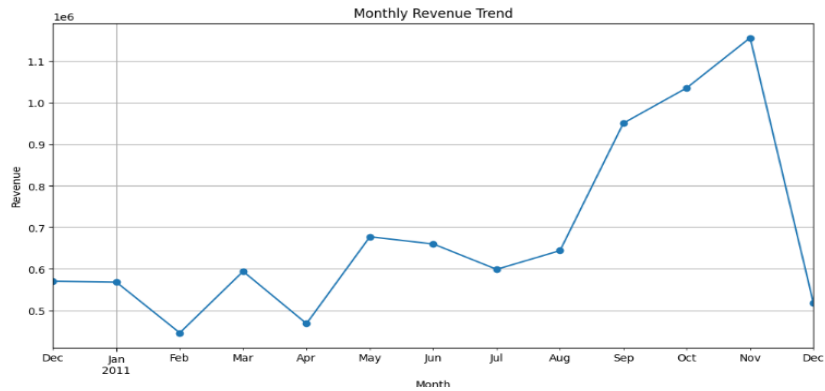
plt.show()
```



```
[38]: df['Month'] = df['InvoiceDate'].dt.to_period('M')
```

```
[39]: monthly_revenue = (
    df.groupby('Month')['TotalAmount']
    .sum()
)
```

```
[40]: plt.figure(figsize=(12,6))
monthly_revenue.plot(marker='o')
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.grid(True)
plt.show()
```

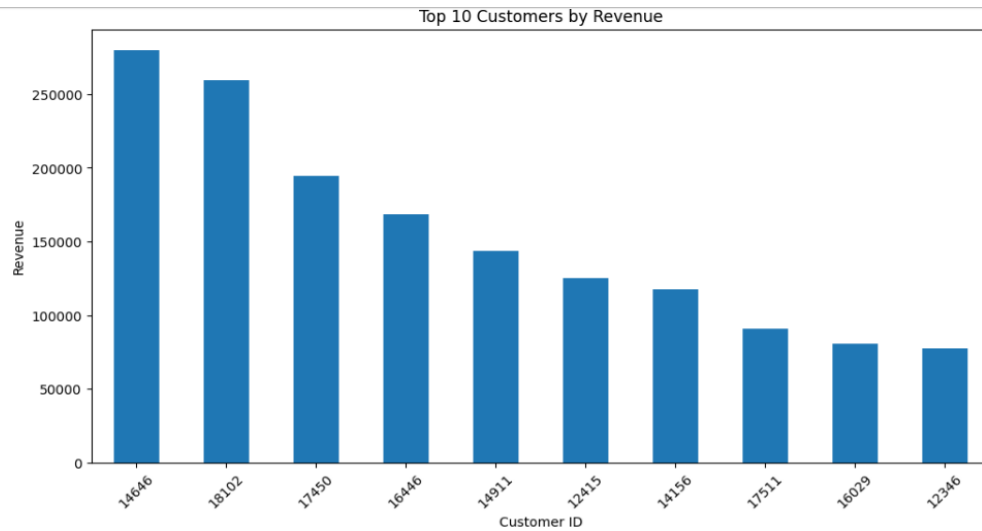


```
[41]: # top customers
customer_revenue = (
    df.groupby('CustomerID')['TotalAmount']
    .sum()
    .sort_values(ascending=False)
)
```

```
[42]: customer_revenue.head(10)
```

```
[42]: CustomerID
14646    280206.02
18102    259657.30
17450    194390.79
16446    168472.50
14911    143711.17
12415    124914.53
14156    117210.08
17511     91062.38
16029     80850.84
12346     77183.60
Name: TotalAmount, dtype: float64
```

```
[43]: plt.figure(figsize=(12,6))
customer_revenue.head(10).plot(kind='bar')
plt.title("Top 10 Customers by Revenue")
plt.xlabel("Customer ID")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.show()
```



```
[44]: # purchase_frequency
purchase_frequency = (
    df.groupby('CustomerID')['InvoiceNo']
      .nunique()
      .sort_values(ascending=False)
)
```

```
[45]: purchase_frequency.head(10)
```

```
[45]: CustomerID
12748    209
14911    201
17841    124
13089     97
14606     93
15311     91
12971     86
14646     73
16029     63
13408     62
```

```
[46]: plt.figure(figsize=(12,6))

purchase_frequency.head(10).plot(kind='bar')

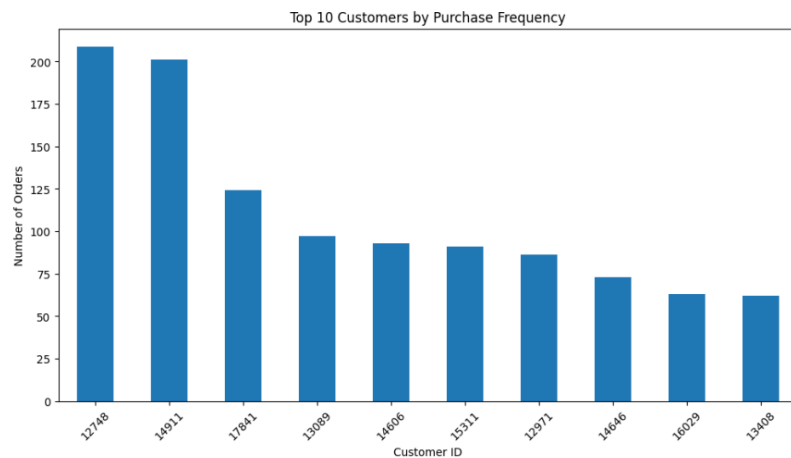
plt.title("Top 10 Customers by Purchase Frequency")

plt.xlabel("Customer ID")

plt.ylabel("Number of Orders")

plt.xticks(rotation=45)

plt.show()
```



```
[47]: snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)
```

```
[48]: recency = (
        df.groupby('CustomerID')['InvoiceDate']
          .max()
      )
```

```
[49]: recency = (snapshot_date - recency).dt.days
```

```
[50]: recency.head()
```

```
[50]: CustomerID
12346    326
12347     2
12348    75
12349    19
12350   310
Name: InvoiceDate, dtype: int64
```

```
[51]: rfm = pd.DataFrame({
        'Recency': recency,
        'Frequency': purchase_frequency,
        'Monetary': customer_revenue
      })
```

```
[52]: rfm.head()
```

```
[52]:
```

	Recency	Frequency	Monetary
CustomerID			
12346	326	1	77183.60
12347	2	7	4310.00
12348	75	4	1797.24
12349	19	1	1757.55
12350	310	1	334.40

```
[53]: rfm['R_Score'] = pd.qcut(
        rfm['Recency'],
        q=5,
        labels=[5,4,3,2,1]
      )
```

```
[54]: rfm['F_Score'] = pd.qcut(
        rfm['Frequency'],
        q=5,
        labels=False,
        duplicates='drop'
      ) + 1
```

```
[55]: rfm['M_Score'] = pd.qcut(
      rfm['Monetary'],
      q=5,
      labels=[1,2,3,4,5]
    )
```

```
[56]: rfm.head()
```

```
[56]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
CustomerID						
12346	326	1	77183.60	1	1	5
12347	2	7	4310.00	5	4	5
12348	75	4	1797.24	2	3	4
12349	19	1	1757.55	4	1	4
12350	310	1	334.40	1	1	2

```
[57]: print(rfm.dtypes)
```

```
Recency      int64
Frequency     int64
Monetary     float64
R_Score      category
F_Score      int64
M_Score      category
dtype: object
```

```
[58]: rfm['R_Score'] = rfm['R_Score'].astype(int)
      rfm['M_Score'] = rfm['M_Score'].astype(int)
```

```
[59]: print(rfm.dtypes)
```

```
Recency      int64
Frequency     int64
Monetary     float64
R_Score      int64
F_Score      int64
M_Score      int64
dtype: object
```

```
[60]: rfm['RFM_Score'] = (
      rfm['R_Score'].astype(str) +
      rfm['F_Score'].astype(str) +
      rfm['M_Score'].astype(str)
    )
```

```
[61]: rfm.head()
```

```
[61]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CustomerID							
12346	326	1	77183.60	1	1	5	115
12347	2	7	4310.00	5	4	5	545
12348	75	4	1797.24	2	3	4	234
12349	19	1	1757.55	4	1	4	414
12350	310	1	334.40	1	1	2	112

```
[62]: def customer_segment(row):
        if row['R_Score'] >= 4 and row['F_Score'] >= 4:
            return 'Champions'
        elif row['R_Score'] >= 3 and row['F_Score'] >= 3:
            return 'Loyal Customers'
        elif row['R_Score'] >= 4 and row['F_Score'] <= 2:
            return 'Potential Loyalists'
        elif row['R_Score'] <= 2 and row['F_Score'] >= 3:
            return 'At Risk'
        else:
            return 'Lost Customers'
```

```
[63]: rfm['Segment'] = rfm.apply(customer_segment, axis=1)
```

```
[64]: rfm.head()
```

```
[64]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score	Segment
CustomerID								
12346	326	1	77183.60	1	1	5	115	Lost Customers
12347	2	7	4310.00	5	4	5	545	Champions
12348	75	4	1797.24	2	3	4	234	At Risk
12349	19	1	1757.55	4	1	4	414	Potential Loyalists
12350	310	1	334.40	1	1	2	112	Lost Customers

```
[65]: segment_count = rfm['Segment'].value_counts()
        print(segment_count)
```

```
Segment
Lost Customers      2099
Potential Loyalists   737
Loyal Customers      721
Champions            578
At Risk              203
Name: count, dtype: int64
```

```
[66]: import matplotlib.pyplot as plt

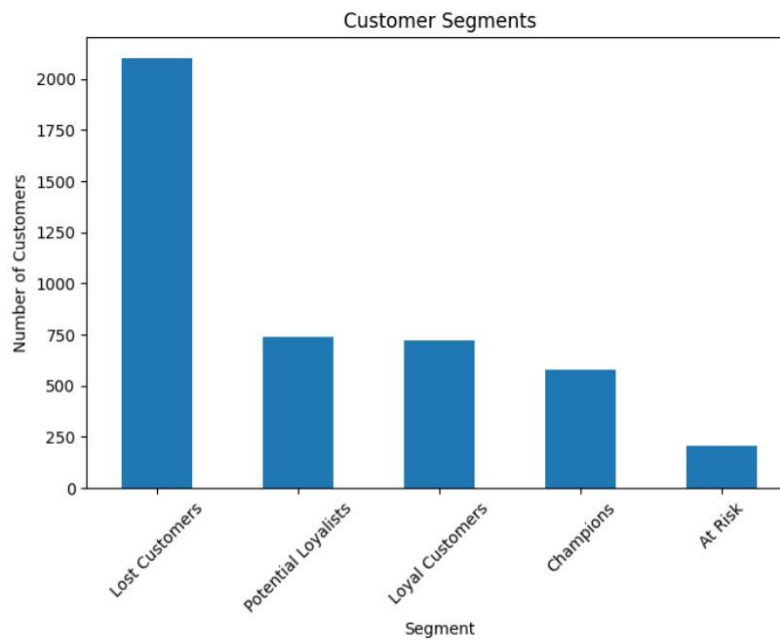
        plt.figure(figsize=(8,5))

        segment_count.plot(kind='bar')

        plt.title("Customer Segments")
        plt.xlabel("Segment")
        plt.ylabel("Number of Customers")

        plt.xticks(rotation=45)

        plt.show()
```



```
[67]: segment_summary = rfm.groupby('Segment').agg(
      Avg_Recency=('Recency', 'mean'),
      Avg_Frequency=('Frequency', 'mean'),
      Avg_Monetary=('Monetary', 'mean'),
      Customer_Count=('Recency', 'count')
    )

segment_summary
```

```
[67]:
```

	Avg_Recency	Avg_Frequency	Avg_Monetary	Customer_Count
Segment				
At Risk	131.339901	5.561576	1826.597438	203
Champions	11.107266	15.335640	8573.205000	578
Lost Customers	159.692234	1.545974	631.435352	2099
Loyal Customers	28.011096	5.342580	2164.387200	721
Potential Loyalists	17.572592	1.956581	916.134532	737

```
[68]: rfm.columns
```

```
[68]: Index(['Recency', 'Frequency', 'Monetary', 'R_Score', 'F_Score', 'M_Score',
      'RFM_Score', 'Segment'],
      dtype='object')
```

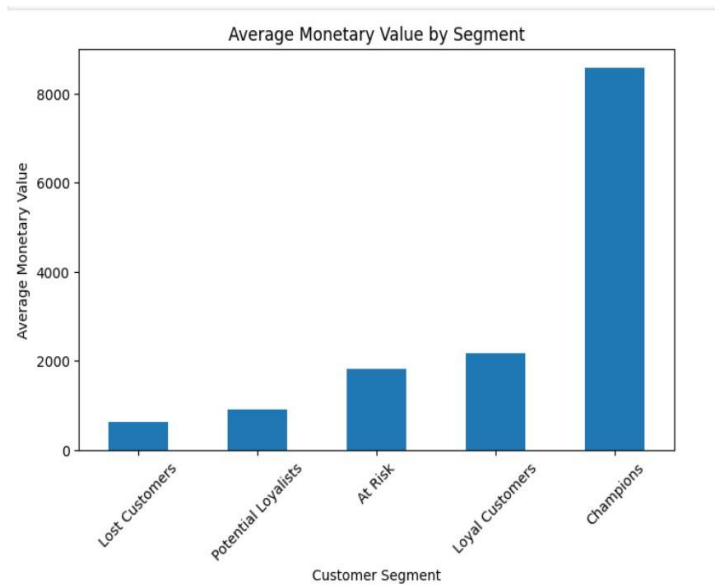
```
[69]: plt.figure(figsize=(8,5))

      segment_summary['Avg_Monetary'].sort_values().plot(kind='bar')

      plt.title("Average Monetary Value by Segment")
      plt.xlabel("Customer Segment")
      plt.ylabel("Average Monetary Value")

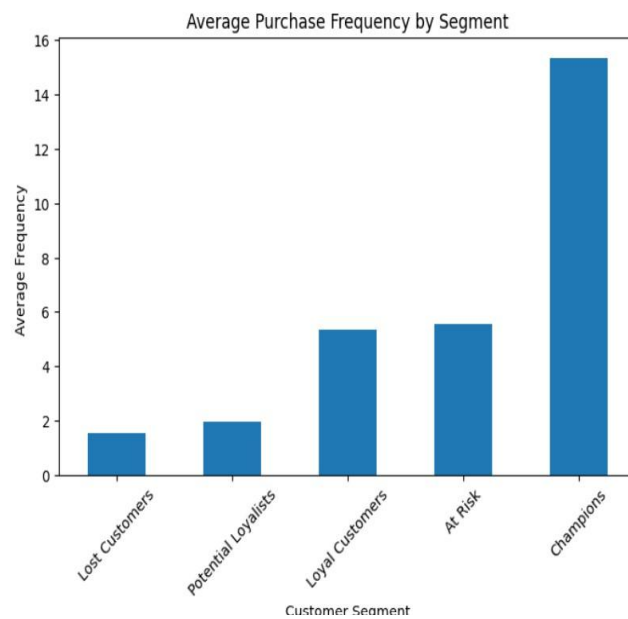
      plt.xticks(rotation=45)

      plt.show()
```



```
[70]: segment_summary.columns  
[70]: Index(['Avg_Recency', 'Avg_Frequency', 'Avg_Monetary', 'Customer_Count'], dtype='object')
```

```
[71]: plt.figure(figsize=(8,5))  
  
segment_summary['Avg_Frequency'].sort_values().plot(kind='bar')  
  
plt.title("Average Purchase Frequency by Segment")  
plt.xlabel("Customer Segment")  
plt.ylabel("Average Frequency ")  
  
plt.xticks(rotation=45)  
  
plt.show()
```



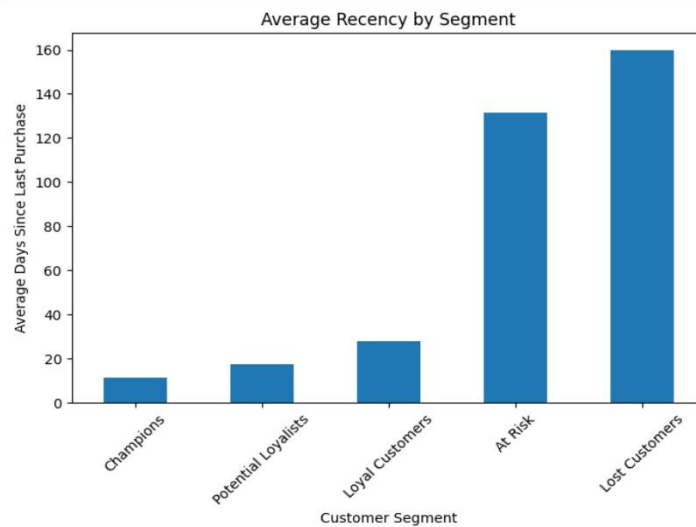
```
[72]: plt.figure(figsize=(8,5))

segment_summary['Avg_Recency'].sort_values().plot(kind='bar')

plt.title("Average Recency by Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Average Days Since Last Purchase")

plt.xticks(rotation=45)

plt.show()
```



```
[73]: segment_revenue = rfm.groupby('Segment')['Monetary'].sum()

print(segment_revenue.sort_values())
```

```
Segment
At Risk          370799.280
Potential Loyalists  675191.150
Lost Customers    1325382.803
Loyal Customers    1560523.171
Champions         4955312.490
Name: Monetary, dtype: float64
```

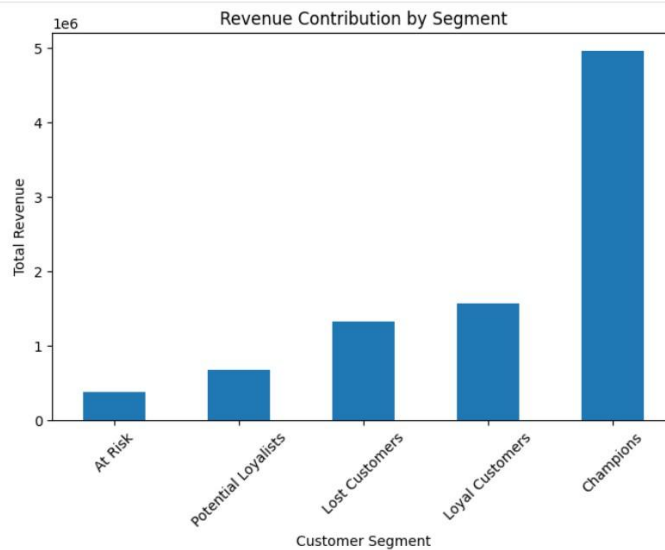
```
[74]: plt.figure(figsize=(8,5))

segment_revenue.sort_values().plot(kind='bar')

plt.title("Revenue Contribution by Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)

plt.show()
```



```
[75]: rfm['Average_Order_Value'] = rfm['Monetary'] / rfm['Frequency']
```

```
[76]: rfm['Average_Order_Value']
```

```
[76]: CustomerID
12346    77183.600000
12347     615.714286
12348     449.310000
12349    1757.550000
12350     334.400000
...
18280     180.600000
18281      80.820000
18282      89.025000
18283     127.845625
18287     612.426667
Name: Average_Order_Value, Length: 4338, dtype: float64
```

```
[77]: #MACHINE LEARNING
# Features (Input)
X = rfm[['Recency', 'Frequency', 'Monetary']]

# Target (Output)
y = rfm['Segment']
```

```
[78]: X.head()
```

```
[78]:
```

	Recency	Frequency	Monetary
CustomerID			
12346	326	1	77183.60
12347	2	7	4310.00
12348	75	4	1797.24
12349	19	1	1757.55
12350	310	1	334.40

```
[79]: y.head()

[79]: CustomerID
12346      Lost Customers
12347      Champions
12348      At Risk
12349  Potential Loyalists
12350      Lost Customers
Name: Segment, dtype: object

[80]: from sklearn.preprocessing import LabelEncoder

# Create LabelEncoder object
label_encoder = LabelEncoder()

# Encode target variable
y_encoded = label_encoder.fit_transform(y)

[81]: print(label_encoder.classes_)

['At Risk' 'Champions' 'Lost Customers' 'Loyal Customers'
 'Potential Loyalists']

[82]: print(y_encoded[:10])

[2 1 0 4 2 3 2 2 4]

[83]: mapping = dict(zip(label_encoder.classes_,
                        label_encoder.transform(label_encoder.classes_)))

print(mapping)

{'At Risk': np.int64(0), 'Champions': np.int64(1), 'Lost Customers': np.int64(2), 'Loyal Customers': np.int64(3), 'Potential Loyalists': np.int64(4)}
```

```
[84]: from sklearn.model_selection import train_test_split

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

```
[85]: from sklearn.ensemble import RandomForestClassifier

# Create the model
rf_model = RandomForestClassifier(
    n_estimators=100, # it include 100 decision tree
    random_state=42
)

# Train the model
rf_model.fit(X_train, y_train)
```

```
[85]: ▼ RandomForestClassifier ⓘ ?
    ► Parameters
    ► Fitted attributes
```

```
[86]: print(rf_model)

RandomForestClassifier(random_state=42)
```

```
[87]: # Predict customer segments
y_pred = rf_model.predict(X_test)
```

```
[88]: print("Actual Labels:")
      print(y_test[:10])

      print("\nPredicted Labels:")
      print(y_pred[:10])

Actual Labels:
[2 3 2 2 2 0 4 2 2 3]

Predicted Labels:
[2 3 2 2 2 0 4 2 2 3]
```

```
[89]: # converting them back to customer segment names.
      predicted_segments = label_encoder.inverse_transform(y_pred)

      print(predicted_segments[:10])

['Lost Customers' 'Loyal Customers' 'Lost Customers' 'Lost Customers'
 'Lost Customers' 'At Risk' 'Potential Loyalists' 'Lost Customers'
 'Lost Customers' 'Loyal Customers']
```

```
[90]: #ACCURACY
      from sklearn.metrics import accuracy_score

      accuracy = accuracy_score(y_test, y_pred)

      print("Accuracy:", accuracy*100,"%")

Accuracy: 100.0 %
```

```
[91]: #PRECISION
      from sklearn.metrics import precision_score

      precision = precision_score(
          y_test,
          y_pred,
          average='weighted'
      )

      print("Precision:", precision*100,"%")

Precision: 100.0 %
```

```
[92]: # RECALL
      from sklearn.metrics import recall_score

      recall = recall_score(
          y_test,
          y_pred,
          average='weighted'
      )

      print("Recall:", recall*100,"%")

Recall: 100.0 %
```

```
[93]: from sklearn.metrics import f1_score

      f1 = f1_score(
          y_test,
          y_pred,
          average='weighted'
      )

      print("F1 Score:", f1)

F1 Score: 1.0
```

```
[94]: #confusion matrices
      from sklearn.metrics import confusion_matrix

      cm = confusion_matrix(y_test, y_pred)

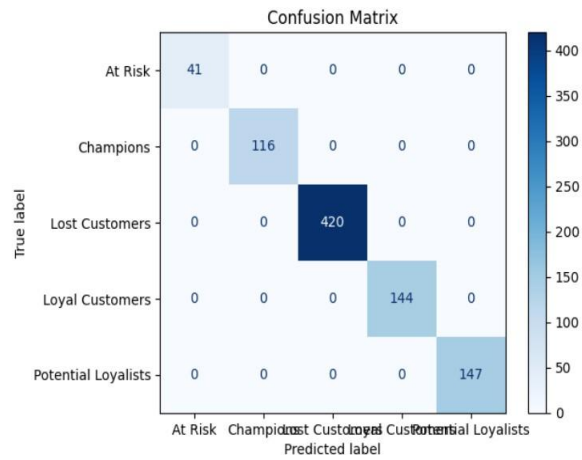
      print(cm)

[[ 41  0  0  0  0]
 [ 0 116  0  0  0]
 [ 0  0 420  0  0]
 [ 0  0  0 144  0]
 [ 0  0  0  0 147]]
```

```
[95]: from sklearn.metrics import ConfusionMatrixDisplay
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=label_encoder.classes_
)
```

```
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```



```
[96]: from sklearn.metrics import classification_report
```

```
print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_
))
```

	precision	recall	f1-score	support
At Risk	1.00	1.00	1.00	41
Champions	1.00	1.00	1.00	116
Lost Customers	1.00	1.00	1.00	420
Loyal Customers	1.00	1.00	1.00	144
Potential Loyalists	1.00	1.00	1.00	147
accuracy			1.00	868
macro avg	1.00	1.00	1.00	868
weighted avg	1.00	1.00	1.00	868

```
[97]: # Get feature importance
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})
```

```
# Sort from highest to lowest
importance = importance.sort_values(
    by='Importance',
    ascending=False
)
```

```
print(importance)
```

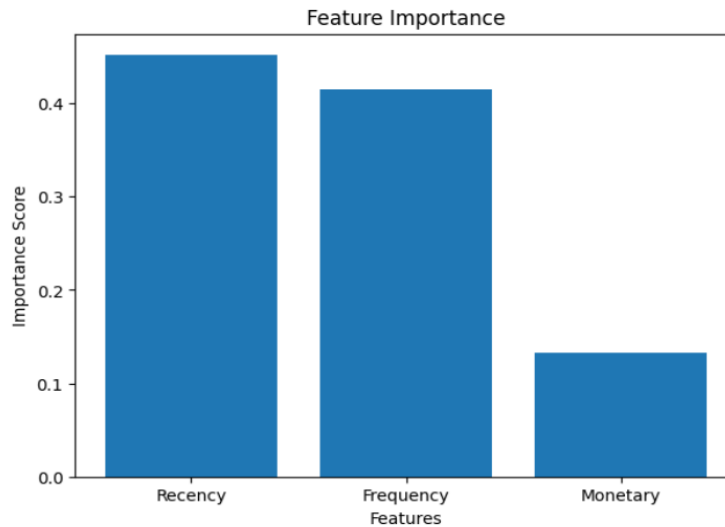
	Feature	Importance
0	Recency	0.452045
1	Frequency	0.415430
2	Monetary	0.132524

```
[98]: plt.figure(figsize=(7,5))

plt.bar(
    importance['Feature'],
    importance['Importance']
)

plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance Score")

plt.show()
```



```
[99]: # New customer data
new_customer = [[8, 12, 4200]]

# Predict encoded label
prediction = rf_model.predict(new_customer)

# Convert back to original segment
predicted_segment = label_encoder.inverse_transform(prediction)

print("Predicted Segment:", predicted_segment[0])

Predicted Segment: Champions
C:\Users\Md Jahid\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning:
e names, but RandomForestClassifier was fitted with feature names
warnings.warn(
```

```
[100]: # New customer data with explicit feature names
new_customer = pd.DataFrame([[8, 12, 4200]], columns=['Recency', 'Frequency', 'Monetary'])
prediction = rf_model.predict(new_customer)
predicted_segment = label_encoder.inverse_transform(prediction)
print("Predicted Segment:", predicted_segment[0])

Predicted Segment: Champions
```

```
[102]: import joblib

# Save the model and preprocessors
joblib.dump(rf_model, "rf_model.pkl")
joblib.dump(label_encoder, "label_encoder.pkl")
joblib.dump(list(X.columns), "feature_columns.pkl")

print("Joblib files saved.")

Joblib files saved.
```

5. References

The following resources were referred to during the development and documentation of this project:

- **UCI Machine Learning Repository** – Online Retail Dataset used for the project.
- **Python Documentation** – Used for understanding Python programming concepts and libraries.
- **Pandas Documentation** – Used for data loading, cleaning, manipulation, and analysis.
- **NumPy Documentation** – Used for numerical calculations and data processing.
- **Matplotlib Documentation** – Used for creating graphs and visualizing data.
- **Seaborn Documentation** – Used for statistical data visualization.
- **Scikit-learn Documentation** – Used for machine learning, Random Forest Classifier, data splitting, and model evaluation.
- **Joblib Documentation** – Used for saving and loading the trained machine learning model.
- **Streamlit Documentation** – Used for developing the interactive application and displaying prediction results.
- **Jupyter Notebook Documentation** – Used for developing and testing the project code.